

taskize

Typst Package

Horizontal Columned Lists

A compact list layout for exercises and multiple-choice questions

Version 0.2.8

Nathan Scheinmann

Contents

- 1. Introduction 3
 - 1.1. Features 3
 - 1.2. Installation 3
 - 1.3. Quick Start 3
- 2. Basic Usage 3
 - 2.1. Column Counts 3
 - 2.2. Auto-Fit Columns 4
 - 2.3. Labels and Formats 6
 - 2.4. Column Spanning 6
 - 2.5. Flow Direction 6
 - 2.6. Adaptive Row Heights 6
 - 2.7. Wrap Zone 7
- 3. Configuration 8
 - 3.1. Global Defaults 8
 - 3.2. Resume Numbering 8
- 4. Parameter Reference 8

1. Introduction

`taskize` is a Typst package for horizontal, columned task lists. Items flow left-to-right across columns, making it ideal for exercises, quizzes, and compact lists that need a tidy grid.

1.1. Features

- Indentation-independent parsing
- Horizontal or vertical flow directions
- Flexible column counts and column spanning
- Auto-fit column selection for the densest no-wrap layout
- Multiple label formats and custom label functions
- Resume numbering across task blocks
- Fine-grained spacing and alignment controls
- Wrap zone to flow rows around reserved overlay space (QR codes, stamps)

1.2. Installation

Import the package in your Typst document:

```
#import "@preview/taskize:0.2.8": tasks
```

1.3. Quick Start

Code

```
#tasks[  
+ First item  
+ Second item  
+ Third item  
+ Fourth item  
]
```

Preview

a) First item	b) Second item
c) Third item	d) Fourth item

2. Basic Usage

2.1. Column Counts

Use `columns:` or shorthand functions (`tasks2`, `tasks3`, `tasks4`) for quick layouts.

Code

```
#tasks(columns: 3)[  
+ Option A  
+ Option B  
+ Option C  
+ Option D  
+ Option E  
+ Option F  
]
```

Preview

a) Option A	b) Option B	c) Option C
d) Option D	e) Option E	f) Option F

Code

```
#tasks3[  
+ Item 1  
+ Item 2  
+ Item 3  
+ Item 4  
+ Item 5  
+ Item 6  
]
```

Preview

a) Item 1	b) Item 2	c) Item 3
d) Item 4	e) Item 5	f) Item 6

2.2. Auto-Fit Columns

Use `columns: "auto-fit"` to select the largest column count that does not introduce a new item wrap or fixed-size content overflow. By default the search goes up to the number of items, so no other option is needed; use `max-columns: only` to cap the search.

Code

```
// No max-columns needed: auto-fit tries up to one
// column per item and picks the densest layout.
#tasks(columns: "auto-fit")[
  + Medium length task with detail
  + Short
  + Short
  + Short
]
```

Preview

a) Medium length task with detail b) Short
c) Short d) Short

By default, `auto-fit` uses `auto-fit-mode: "fill"`: span items may use their full row while later rows remain dense. Use `auto-fit-mode: "uniform"` when every item must fit one ordinary column; in that mode, the column count is the largest value where no item wraps when measured at one-column width, so a wide span item can force the whole block to one column. If an item already wraps in the one-column layout, `auto-fit` keeps one column, because no no-wrap multi-column layout exists.

A wide item auto-spans to fill the row in `"fill"` mode, while `"uniform"` keeps every item in one column and falls back to fewer columns when the widest item no longer fits.

Code

```
// fill: wide instruction auto-spans; cities pack
// into 4 cols.
#tasks(columns: "auto-fit", auto-fit-mode: "fill",
  max-columns: 4)[
  + Choose the correct answer for each question
  below.
  + Paris
  + London
  + Berlin
  + Rome
]
```

Preview

"fill", max 4 — instruction spans full row; answers in 4 cols:

a) Choose the correct answer for each question below.
b) Paris c) London d) Berlin e) Rome

Code

```
// uniform: instruction too wide for one column at 2+
// cols;
// auto-fit falls back to a single column.
#tasks(columns: "auto-fit", auto-fit-mode: "uniform",
  max-columns: 4)[
  + Choose the correct answer for each question
  below.
  + Paris
  + London
  + Berlin
  + Rome
]
```

Preview

"uniform", max 4 — instruction forces 1-column layout:

a) Choose the correct answer for each question below.
b) Paris
c) London
d) Berlin
e) Rome

With medium-length items (shorter than the full row but still wider than one narrow column), `"fill"` auto-spans and keeps the maximum column count while `"uniform"` backs off to the column count where the item fits in one column slot. The same difference applies with 2 or 3 columns.

Code

```
// fill, max 2: medium item auto-spans both cols (=
full
// row); two short items fill row 2 side by side.
#tasks(columns: "auto-fit", auto-fit-mode: "fill",
max-columns: 2)[
+ Medium length task with detail
+ Short
+ Short
]
```

Preview

"fill", max 2 — medium item spans full row; shorts in 2 cols:

- a) Medium length task with detail
- b) Short
- c) Short

Code

```
// uniform, max 2: medium item too wide for one col
at 2
// cols; auto-fit falls back to a single column.
#tasks(columns: "auto-fit", auto-fit-mode: "uniform",
max-columns: 2)[
+ Medium length task with detail
+ Short
+ Short
]
```

Preview

"uniform", max 2 — medium item forces 1-column layout:

- a) Medium length task with detail
- b) Short
- c) Short

Code

```
// fill, max 3: medium item auto-spans; 3 cols.
#tasks(columns: "auto-fit", auto-fit-mode: "fill",
max-columns: 3)[
+ Medium length task
+ Short
+ Short
+ Short
]
// uniform, max 3: medium item must fit one column;
// backs off to 2 columns where it does.
#tasks(columns: "auto-fit", auto-fit-mode: "uniform",
max-columns: 3)[
+ Medium length task
+ Short
+ Short
+ Short
]
```

Preview

"fill" vs "uniform", max 3 — fill gives 3 cols, uniform gives 2:

- a) Medium length task
 - b) Short
 - c) Short
 - d) Short
-
- a) Medium length task
 - b) Short
 - c) Short
 - d) Short

Code

```
// fill, max 4: medium item auto-spans; 4 cols.
#tasks(columns: "auto-fit", auto-fit-mode: "fill",
max-columns: 4)[
+ Medium length task
+ Short
+ Short
+ Short
]
// uniform, max 4: medium item must fit one column;
// backs off to 2 columns where it does.
#tasks(columns: "auto-fit", auto-fit-mode: "uniform",
max-columns: 4)[
+ Medium length task
+ Short
+ Short
+ Short
]
```

Preview

"fill" vs "uniform", max 4 — fill gives 4 cols, uniform gives 2:

- a) Medium length task
 - b) Short
 - c) Short
 - d) Short
-
- a) Medium length task
 - b) Short
 - c) Short
 - d) Short

2.3. Labels and Formats

Code

```
#tasks(label: "A") [+ Alpha + Beta + Gamma]
#tasks(label: "(1)") [+ One + Two + Three]
#tasks(label: n => "Q" + str(n) + ":") [
  + What is 2 + 2?
  + What is the capital of France?
  + What color is the sky?
]
```

Preview

A) Alpha + Beta + B) Beta
Gamma
C) Gamma
(1) One + Two + Three (2) Two
(3) Three
Q1: What is 2 + 2? Q2: 2?
Q3: What is the capital of France? Q4: What color is the sky?

2.4. Column Spanning

Use (N) to span N columns or () to span all columns.

Code

```
#tasks(columns: 3) [
  + (2) Long statement spanning two columns
  + Short
  + () Full width note across all columns
  + Another
]
```

Preview

a) Long statement spanning two columns b) Short
c) Full width note across all columns
d) Another

2.5. Flow Direction

Horizontal is default. Use flow: "vertical" to fill columns top-to-bottom.

Code

```
#tasks(columns: 2, flow: "vertical") [
  + a
  + b
  + c
  + d
  + e
  + f
]
```

Preview

a) a d) d
b) b e) e
c) c f) f

2.6. Adaptive Row Heights

Rows containing tall inline math (display fractions, matrices, big operators) normally report a height based on the font's text edges, not the actual ink — so the math bleeds into the row-gutter and rows look glued together. Pass row-gutter: "auto" (or "adaptive") to size rows by their true ink: the configured numeric gutter (default 0.6em) becomes the real visual gap, and rows with ordinary content keep exactly the same spacing as before.

Code

```
#show math.frac: it => math.display(it)
// ink bleeds into the gutter:
#tasks(columns: 2)[
  + $1/2$
  + $3/4$
  + $5/6$
  + $7/8$
]
// rows sized to true ink:
#tasks(columns: 2, row-gutter: "auto")[
  + $1/2$
  + $3/4$
  + $5/6$
  + $7/8$
]
```

Preview

a) $\frac{1}{2}$	b) $\frac{3}{4}$
c) $\frac{5}{6}$	d) $\frac{7}{8}$
a) $\frac{1}{2}$	b) $\frac{3}{4}$
c) $\frac{5}{6}$	d) $\frac{7}{8}$

It can be made the document default with `#tasks-setup(row-gutter: "adaptive")`; an explicit numeric `row-gutter` on a `tasks` call still overrides it.

2.7. Wrap Zone

Reserve a rectangular zone at the top right of the next `#tasks` call so its rows flow around overlay content placed there independently — a QR code, a stamp, a logo. Rows that would overlap the zone render in a narrowed column beside it; once enough rows have passed to clear the zone's height, the remaining rows return to full width. Only horizontal flow (the default) supports this — vertical flow ignores a pending zone, since narrowing rows would reorder the numbering.

Set the zone with the `taskize-wrap-zone` state before the `#tasks` call and place the overlay content yourself (`tasks` only reserves the space, it does not draw anything there):

Code

```
#taskize-wrap-zone.update((width: 2.4cm, height:
2.4cm))
#place(top + right, box(
  width: 2.4cm, height: 2.4cm,
  stroke: 0.5pt + luma(60%),
)[QR code])

#tasks(columns: 2)[
  + $2 + 3$
  + $5 - 1$
  + $4 \times 2$
  + $9 \div 3$
  + $7 + 6$
  + $12 - 4$
]
```

Preview

a) $2 + 3$	b) $5 - 1$
c) 4×2	d) $9 \div 3$
e) $7 + 6$	f) $12 - 4$

QR code

The state is consumed automatically: the first `#tasks` call that flows around it resets it to `none`, so later calls in the same document are unaffected by a zone set earlier. Two ways to override the shared state on a single call:

Name	Type	Default	Description
wrap-zone (parameter of <code>tasks()</code>)	auto/none/dictionary	auto	<code>auto</code> honors the shared <code>taskize-wrap-zone</code> state (the common case); <code>none</code> ignores a pending zone even if one is set; (<code>width: <length></code> , <code>height: <length></code>) reserves an explicit zone for this call only, without reading or clearing the shared state

<code>taskize-wrap-zone</code> (state)	<code>none/dictionary</code>	<code>none</code>	Shared state read by <code>tasks()</code> when its own <code>wrap-zone</code> parameter is <code>auto</code> . Set it with <code>#taskize-wrap-zone.update((width:, height:))</code> before the call; it is reset to <code>none</code> once a <code>#tasks</code> call consumes it. This is the contract wrapper packages (e.g. <code>exercise-bank</code>) use to reserve space for content they overlay themselves
---	------------------------------	-------------------	---

3. Configuration

3.1. Global Defaults

Use `tasks-setup()` to set defaults for the rest of the document.

Code

```
#tasks-setup(
  columns: 3,
  label-format: "1)",
  column-gutter: 1.2em,
  row-gutter: 0.7em,
  label-weight: "bold",
)

#tasks[
+ Item 1
+ Item 2
+ Item 3
+ Item 4
+ Item 5
+ Item 6
]
```

Preview

1) Item 1	2) Item 2	3) Item 3
4) Item 4	5) Item 5	6) Item 6

3.2. Resume Numbering

Code

```
#tasks[
+ First
+ Second
+ Third
]

#tasks(resume: true)[
+ Fourth
+ Fifth
]

#tasks-reset()
#tasks[
+ Back to one
+ Two again
]
```

Preview

1) First	2) Second	3) Third
4) Fourth	5) Fifth	
1) Back to one	2) Two again	

4. Parameter Reference

The `tasks()` function supports the following parameters:

Parameter	Type	Default	Description
-----------	------	---------	-------------

columns	int/string	2	Number of columns, or " auto-fit " for the largest no-wrap count
label	string/function	"a")	Label format shorthand
label-format	string/function	"a")	Explicit label format
start	int	1	Starting number
resume	bool	false	Continue numbering from previous tasks
column-gutter	length	1em	Space between columns
row-gutter	length/string	0.6em	Space between rows. " auto " (or " adaptive ") sizes rows to the true ink of their content — tall inline math (display fractions, matrices) no longer bleeds into the gutter — while keeping the configured numeric gutter as the visual gap
max-columns	int/auto	auto	Maximum column count tested by auto-fit; auto means up to the number of items
auto-fit-mode	string	"fill"	" fill " measures spans at their rendered width; " uniform " requires every item to fit one ordinary column
auto-fit-tolerance	length	0.5pt	Measurement tolerance for auto-fit wrap detection
label-width	auto/length	auto	Reserved width for labels
label-align	alignment	right	Label alignment
label-baseline	string/length	"center"	Vertical alignment of labels. With " center " (default), inline content (text, math) shares the same paragraph line as the label for true baseline alignment. Use " top ", " bottom ", or a length offset to override.
label-weight	string	"regular"	Font weight for labels
indent-after-label	length	0.4em	Space after label
indent	length	0pt	Left indentation of the block
above	length	0.5em	Space before the block
below	length	0.5em	Space after the block
flow	string	"horizontal"	"horizontal" or "vertical" flow
wrap-zone	auto/none/dictionary	auto	auto honors the shared taskize-wrap-zone state; none ignores a pending zone; (width: , height:) sets an explicit zone for this call only. See Wrap Zone